Paradigmas de Programação — Aula 09: Modelo de atores com Elixir

Exemplo: processador de pagamentos

O problema

Pagamento	Valor	Latência	Desfecho no gateway
pg-1	R$ 120,00	100 ms	aprovado
pg-2	R$ 89,90	150 ms	*timeout* na 1ª tentativa; aprovado na 2ª
pg-3	R$ 1.500,00	80 ms	recusado: saldo insuficiente
pg-4	R$ 45,00	200 ms	aprovado
pg-5	R$ 310,00	120 ms	aprovado

- Em sequência: 100 + 150 + 80 + 200 + 120 + 150 (retry) = **800 ms**
- Pagamentos **independentes**, que passam o tempo **esperando** o gateway
- A falha da pg-2 não pode impedir nem atrasar os outros quatro
- Latência simulada com `Process.sleep`; num sistema real, uma requisição HTTP

A estrutura

- **Supervisor** `:one_for_one` com seis filhos: o `Gateway` e um `Pagamento` por pagamento
- **`Gateway`** (GenServer, registrado por nome): conta as tentativas de cada pagamento
  - o processo reiniciado começa com estado inicial e não lembraria que já tentou
- **`Pagamento`** (GenServer de vida curta): pede a tentativa, espera, envia `{:resultado, ...}` ao `main` e termina
  - aprovado ou recusado: `{:stop, :normal, estado}`
  - falha transitória: `raise`, sem `try` / `rescue`
  - `restart: :transient`: reinicia só quem terminou de forma anormal
  - trabalho em `handle_continue/2`: o `init/1` retorna logo, e o Supervisor não espera um pagamento para iniciar o próximo

Onde fica o `Process.sleep`

```
def autorizar(pagamento) do
  tentativa = GenServer.call(__MODULE__, {:tentativa, pagamento.id})  # responde na hora
  Process.sleep(pagamento.latencia)                                   # no processo de quem chama
  # ... decide o desfecho
end
```

- Um ator trata **uma mensagem por vez**
- Com o `sleep` dentro do `handle_call`, o `Gateway` atenderia os cinco **em fila**: cerca de 800 ms
- A propriedade que elimina a corrida é a que torna um ator compartilhado um **gargalo**
- Regra: o ator compartilhado faz só o trabalho que precisa do estado dele; o demorado fica com quem pediu

A saída

```
  pg-3  recusado: saldo insuficiente  (86 ms)
  pg-1  aprovado  R$ 120.00  (105 ms)
  pg-5  aprovado  R$ 310.00  (123 ms)
  [log] pg-2 caiu na tentativa 1: timeout do gateway -> supervisor reinicia
  pg-4  aprovado  R$ 45.00  (204 ms)
  pg-2  aprovado  R$ 89.90  (317 ms, 2a tentativa)
tempo total: 317 ms (em sequencia seriam 800 ms)
```

- Resultados na ordem das **latências**, não na da lista; tempos variam alguns ms entre execuções

A linha do tempo

```
ms      0    50   100  150  200  250  300
pg-3    ========                      recusado
pg-1    ==========                    aprovado
pg-5    ============                  aprovado
pg-2    ===============X              tentativa 1: timeout, cai
pg-2                    ===============  tentativa 2: aprovado
pg-4    ====================          aprovado
```

- Em 150 ms a pg-2 cai; o Supervisor inicia um processo novo, que pede a tentativa 2 ao `Gateway`
- A pg-4, ainda esperando quando a pg-2 cai, não é afetada
- O último a terminar não é o de maior latência (pg-4, 200 ms), e sim a pg-2: 150 + 150 = 300 ms

O que o exemplo mostra

- **Isolamento de falha**: não há estado compartilhado que a queda da pg-2 pudesse deixar pela metade; `:one_for_one` reinicia só ela
- **Erro de negócio não é falha**: a recusa da pg-3 é valor de retorno e término `:normal`; o *let it crash* vale para a falha transitória
- **Defeito determinístico**: cai a cada tentativa, e o reinício não resolve
  - padrão `max_restarts: 3` em `max_seconds: 5`: na 4ª queda o Supervisor encerra os filhos e termina
  - o código não demonstra o caso, para manter a saída estável
- **Tempo ≈ o do mais lento, contando o retry**: cerca de 317 ms medidos, contra 800 ms em sequência
- **Como na cotação da aula 08**: esperas sobrepostas; a diferença é que a falha é tratada por um processo que só vigia

Os três modelos concorrentes

	Java: threads e trava	Go: gorrotinas e canais	Elixir: atores
Onde fica o estado	memória compartilhada	em qualquer gorrotina; por convenção, com um dono	privado de cada processo
Como se comunica	lê e escreve a mesma memória, sob `synchronized`	envia e recebe por canal; sem *buffer*, sincroniza	mensagem assíncrona ao PID, na caixa de correio
O que acontece numa falha	a thread termina; o estado compartilhado pode ficar pela metade	`panic` não recuperado encerra o programa inteiro	o processo cai sozinho; o Supervisor o reinicia com estado inicial